Task 1:

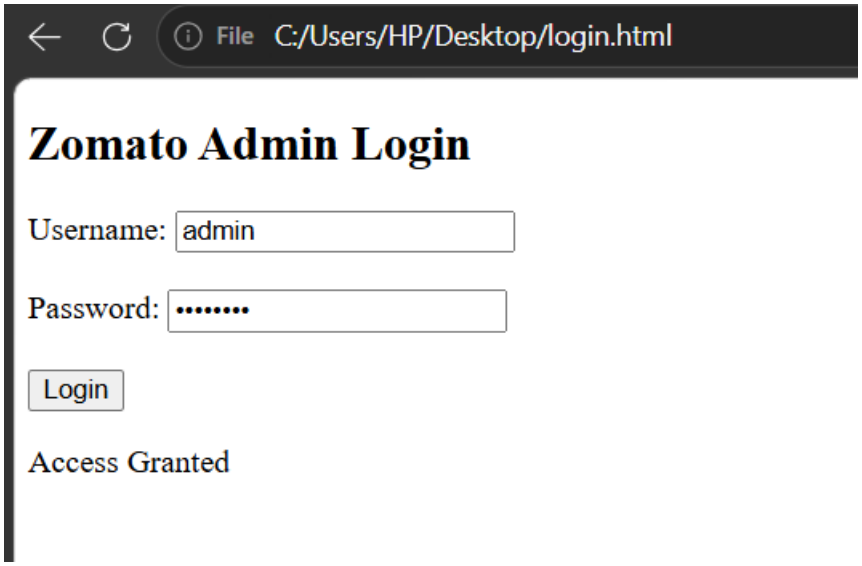
Create a simple login form for a mock Zomato admin panel that asks for username and password, then displays 'Access Granted' only if both fields match hardcoded admin credentials.

Answer:

```
<!DOCTYPE html>
<html>
<head>
   <title>Zomato Admin Login</title>
</head>
<body>
   <h2>Zomato Admin Login</h2>
  <form onsubmit="checkLogin(event)">
      <label>Username:</label>
      <input type="text" id="username" required>
      <br><br>
 <label>Password:</label>
      <input type="password" id="password" required>
      <br><br>
 <button type="submit">Login</button>
   </form>
 <p id="message"></p>
<script>
      function checkLogin(event) {
         event.preventDefault();

         const username =
            document.getElementById("username").value;
         const password =
            document.getElementById("password").value;
```

```
        if (username === "admin" &&
            password === "admin123") {
            document.getElementById("message").textContent =
                "Access Granted";
        } else {
            document.getElementById("message").textContent =
                "Access Denied";
        }
    }
  </script>
</body>
</html>
```



Task 2

Write a Python function check_access(user_role, resource) that returns True if the user_role is allowed to access the resource, otherwise False. Test it with roles like 'customer', 'restaurant_owner', and resources like 'menu_edit', 'order_history', 'admin_dashboard'.

Answer:

```
def check_access(user_role, resource):
    permissions = {
        "customer": ["menu_view", "order_history"],
        "restaurant_owner": [
            "menu_view",
            "menu_edit",
            "order_history"
        ],
        "admin": [
            "menu_view",
            "menu_edit",
            "order_history",
            "admin_dashboard"
        ]
    }

    return resource in permissions.get(user_role, [])



# Test cases
print(check_access("customer", "order_history"))
print(check_access("customer", "admin_dashboard"))
print(check_access("restaurant_owner", "menu_edit"))
print(check_access("restaurant_owner", "admin_dashboard"))
print(check_access("admin", "admin_dashboard"))
```

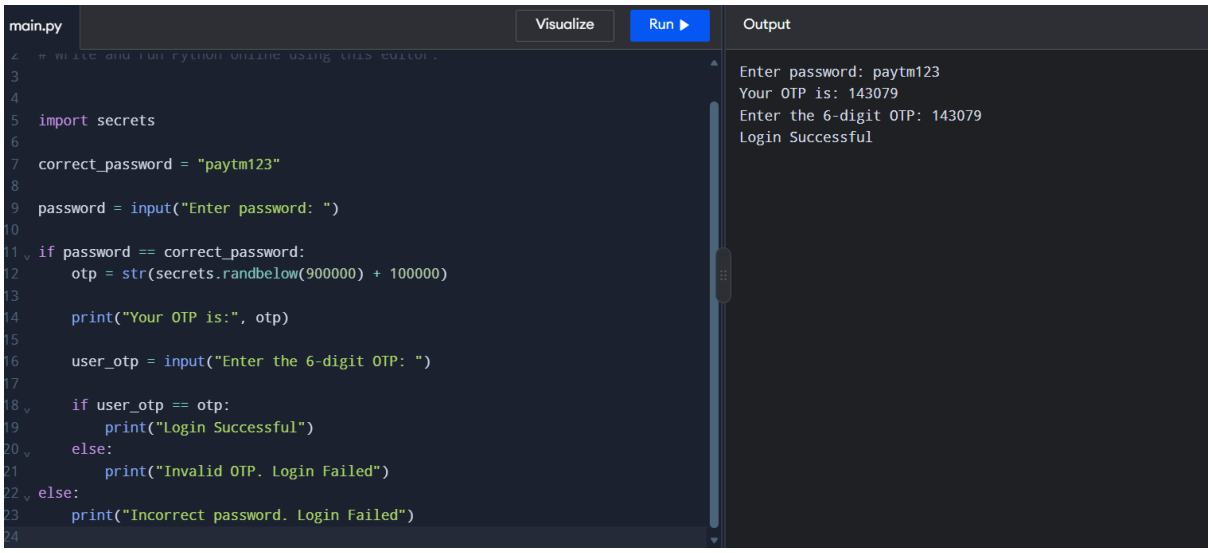
```
True
False
True
False
True
```

Task 3:

Simulate a two-factor authentication flow like Paytm: after a user enters the correct password, generate a 6-digit OTP and display it in the console. Ask the user to input the OTP and display 'Login Successful' only if it matches.

Answer:

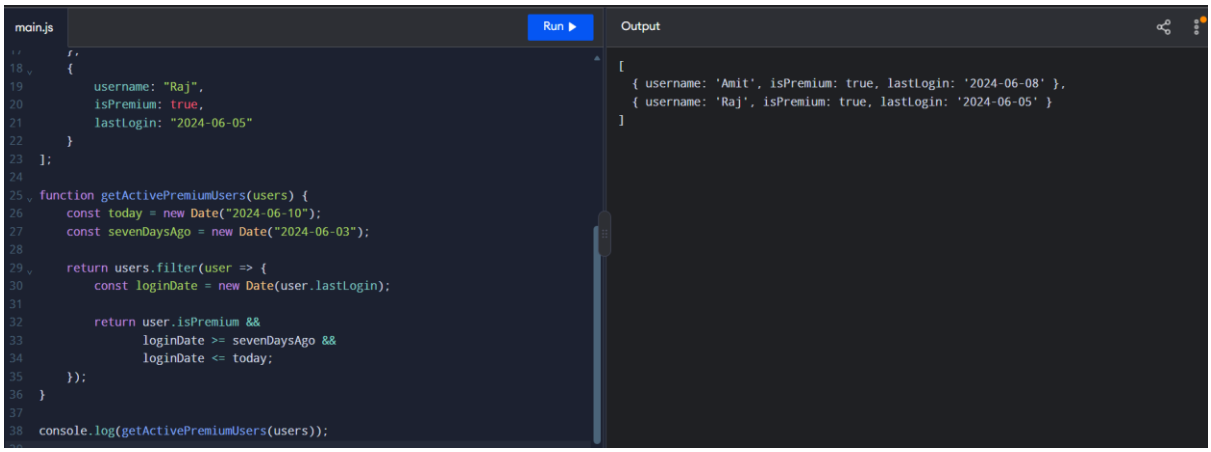
```
import secrets
correct_password = "paytm123"
password = input("Enter password: ")
if password == correct_password:
    otp = str(secrets.randbelow(900000) + 100000)
    print("Your OTP is:", otp)
    user_otp = input("Enter the 6-digit OTP: ")
 if user_otp == otp:
        print("Login Successful")
    else:
        print("Invalid OTP. Login Failed")
else:
    print("Incorrect password. Login Failed")
```



Task 4:

Given an array of user objects with properties: username, isPremium, and lastLogin, write a function that returns only those users who are premium and have logged in within the last 7 days.

Answer:



Task 5:

Use ChatGPT to generate a list of 5 common access control mistakes developers make in real-world apps like Instagram or Flipkart, and for each, write a one-line fix or prevention tip.

Answer:

No.	Access control mistake	One-line prevention tip
1	Broken Access Control: A customer accesses an admin page without permission.	Check the user's role and permissions on every protected server endpoint.
2	IDOR (Insecure Direct Object Reference): A user changes an order ID in the URL to view another customer's order.	Verify that the requested record belongs to the logged-in user or that the user has explicit permission.
3	Privilege Escalation: A customer changes their role to administrator.	Never trust roles supplied by the client; determine permissions from trusted server-side data.
4	Missing Authorization Checks: An API allows unauthorized users to edit a menu or cancel an order.	Enforce authorization checks for every sensitive API operation.
5	Weak Session Management: An attacker reuses a stolen or old session token.	Use secure session cookies, rotate session IDs after login, and expire or revoke sessions appropriately.

Conclusion: Access control ensures users can access only the resources and actions permitted for their roles. Strong authentication, server-side authorization, and secure session management help protect real-world applications such as Instagram, Zomato, and Flipkart.